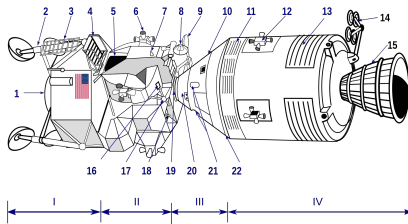


Welcome to
ASEN 3502
Computational Methods

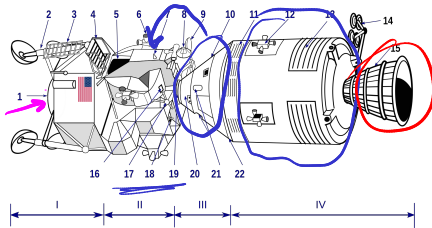
Apollo 13



I-II: lunar module (descent, ascent stages); III: command module; IV: service module.

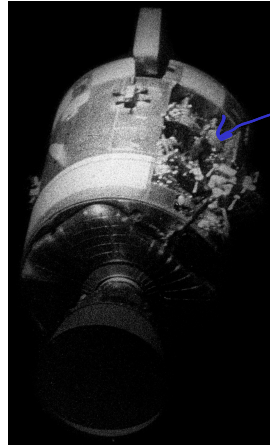
Popadius, Wikimedia Commons, CC0.

Apollo 13



I-II: lunar module (descent, ascent stages); III: command module; IV: service module.

Popadius, Wikimedia Commons, CC0.



The service module after separation, with one panel blown off. NASA (scan by Kipp Teague), public domain.

Direct return

NASA-S-66-4979 MAY 25

DIRECT RETURN MODE FOR TL COAST

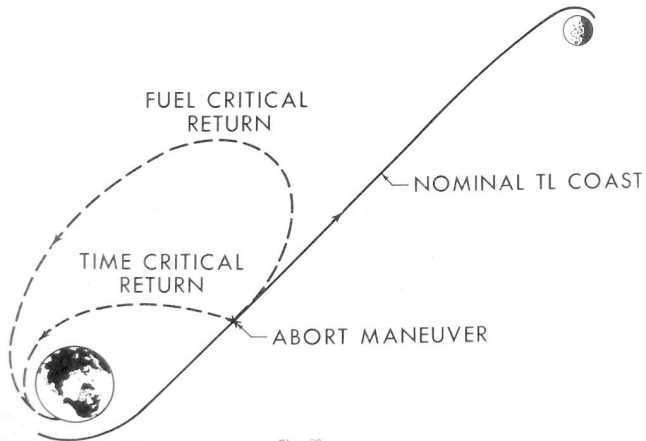
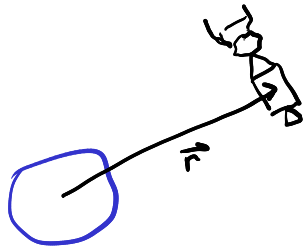


Fig. 20

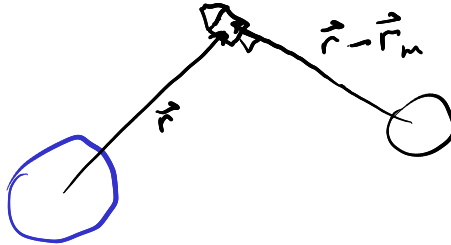
R. L. Berry, *Apollo Lunar Landing Mission Symposium*, NASA MSC, June 1966. Public domain, via Wikimedia

Commons.



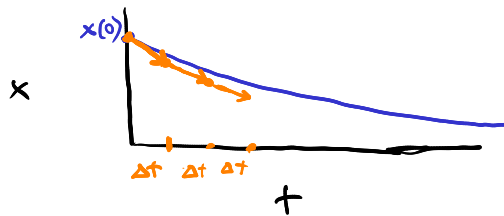
$$\ddot{\vec{r}} = -\frac{\mu_e}{r^2} \frac{\vec{r}}{r} \rightarrow \text{elliptical hyperbolic}$$

$$\mu_e = G m_e$$



$$\ddot{\vec{r}} = -\frac{\mu_e}{r^2} \frac{\vec{r}}{r} - \frac{\mu_m}{\|\vec{r} - \vec{r}_m\|^2} \frac{\vec{r} - \vec{r}_m}{\|\vec{r} - \vec{r}_m\|}$$

no easy analytical solution



$$\dot{x} = -ax$$

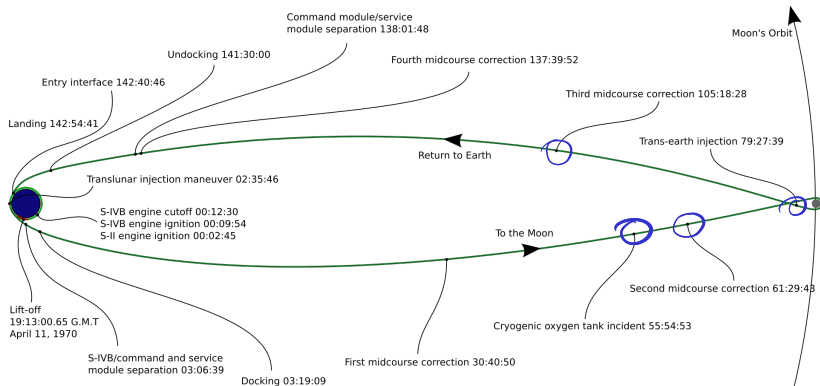
Euler's Method

$$\text{Error} = O(\Delta t)$$

Runge-Kutta

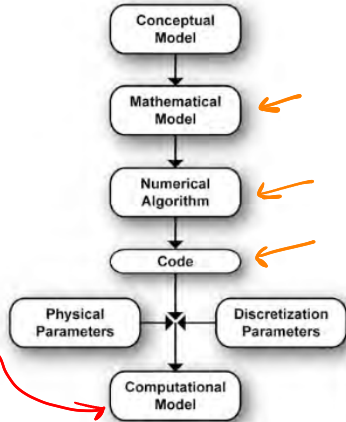
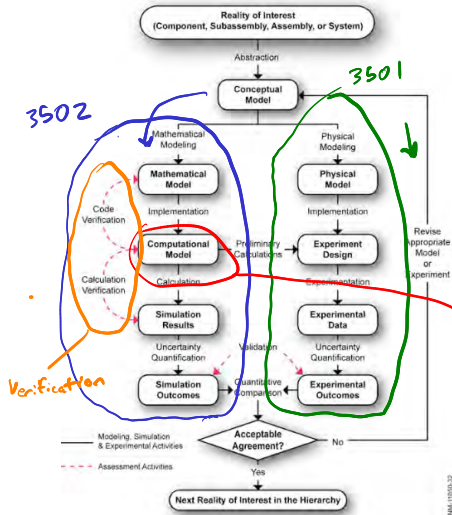
$$\text{Error} = O(\Delta t^4)$$

What Apollo 13 actually flew



AndrewBuck, Wikimedia Commons, CC BY-SA 4.0. Source for figures: Apollo 13 Mission Report, p. 3-2.

ASME V&V 10

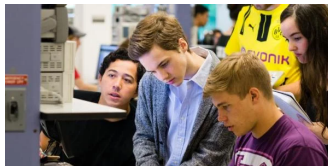


Your code is not done until you verify it with tests

asen3502.com

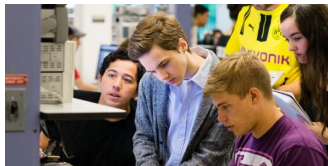
Our Assumptions

- The material is challenging, and can take multiple engagements to understand fully



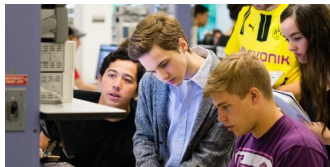
Our Assumptions

- ▶ The material is challenging, and can take multiple engagements to understand fully
- ▶ A student who comes to my office or asks a question wants to learn



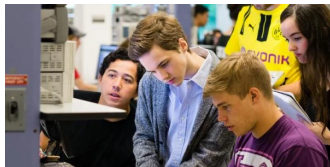
Our Assumptions

- ▶ The material is challenging, and can take multiple engagements to understand fully
- ▶ A student who comes to my office or asks a question wants to learn
- ▶ It may take multiple ways of explaining a concept before a student understands



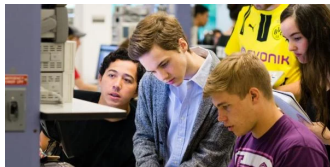
Our Assumptions

- ▶ The material is challenging, and can take multiple engagements to understand fully
- ▶ A student who comes to my office or asks a question wants to learn
- ▶ It may take multiple ways of explaining a concept before a student understands
- ▶ We must work together to help you find the way to understand



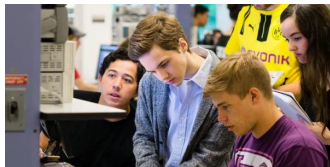
Our Assumptions

- ▶ The material is challenging, and can take multiple engagements to understand fully
- ▶ A student who comes to my office or asks a question wants to learn
- ▶ It may take multiple ways of explaining a concept before a student understands
- ▶ We must work together to help you find the way to understand
- ▶ Taking notes in class and doing the homework yourselves are critical steps to understanding



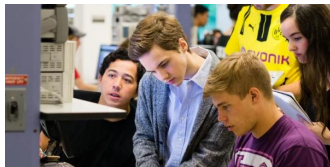
Our Assumptions

- ▶ The material is challenging, and can take multiple engagements to understand fully
- ▶ A student who comes to my office or asks a question wants to learn
- ▶ It may take multiple ways of explaining a concept before a student understands
- ▶ We must work together to help you find the way to understand
- ▶ Taking notes in class and doing the homework yourselves are critical steps to understanding
- ▶ I am trying to bring you to the point of understanding, not just give you the answer



Our Assumptions

- ▶ The material is challenging, and can take multiple engagements to understand fully
- ▶ A student who comes to my office or asks a question wants to learn
- ▶ It may take multiple ways of explaining a concept before a student understands
- ▶ We must work together to help you find the way to understand
- ▶ Taking notes in class and doing the homework yourselves are critical steps to understanding
- ▶ I am trying to bring you to the point of understanding, not just give you the answer
- ▶ We will have fun this semester



Today

- Linear Systems of Equations
- Gaussian Elim.
- Partial Pivoting

$$\begin{aligned} 2x_1 + x_2 &= 4 \\ -x_1 + x_2 &= 1 \end{aligned}$$

Unknowns: $x_1, \dots, x_n$

Linear Eqn.: $\sum_{j=1}^n a_{ij} x_j = b_i$

no nonlinear functions of x_j
 $x^2, e^x, \log(x), \sin(x)$ etc.

System of m
linear equations

$$\sum_{j=1}^n a_{ij} x_j = b_i \quad i = 1, \dots, m$$

$$\begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{m1} & & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix}$$

Book: $[A] \{x\} = \{b\}$

Handwriting: $A \vec{x} = \vec{b}$

for now $m = n$

HWO $\Rightarrow A \cdot x$

Lab

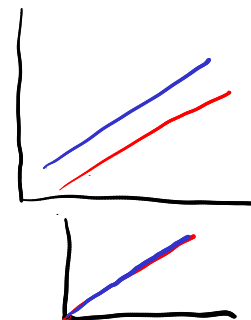
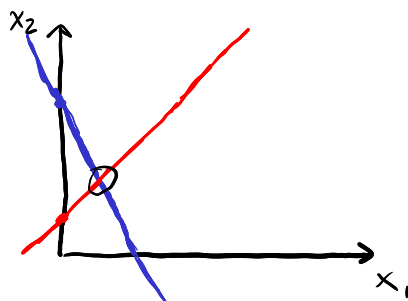
$Ax = b$
 Known $\uparrow$ Unknowns $\uparrow$ Known

$$\begin{aligned} 2x_1 + x_2 &= 4 \\ x_2 &= -2x_1 + 4 \end{aligned}$$

$$-x_1 + x_2 = 1$$

$$x_2 = x_1 + 1$$

$$\boxed{x_1 = 1, x_2 = 2}$$



Example:

$$\begin{bmatrix} 3 & -1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 6 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$3x_1 - x_2 = 0 \rightarrow 3x_1 - 3 = 0$$

$$\begin{aligned} 2x_2 &= 6 \\ x_2 &= 3 \end{aligned}$$

$$\boxed{x_1 = 1}$$

$$2x_1 + 3x_2 + 4x_3 = 19 \quad (1)$$

$$4x_1 + 11x_2 + 14x_3 = 55 \quad (2)$$

$$2x_1 + 8x_2 + 17x_3 = 50 \quad (3)$$

$$\begin{bmatrix} 2 & 3 & 4 & : & 19 \\ 4 & 11 & 14 & : & 55 \\ 2 & 8 & 17 & : & 50 \end{bmatrix}$$

$$\begin{aligned} & -2(1) \\ & -1(3) \end{aligned}$$

$$\begin{bmatrix} 2 & 3 & 4 & : & 19 \\ 0 & 5 & 6 & : & 17 \\ 0 & 5 & 13 & : & 31 \end{bmatrix}$$

$$-1(2)$$

$$\begin{bmatrix} 2x_1 + 3x_2 + 4x_3 = 19 \\ 0 \quad 5x_2 + 6x_3 = 17 \\ 0 \quad 0 \quad 7x_3 = 14 \end{bmatrix}$$

$$x_3 = 2$$

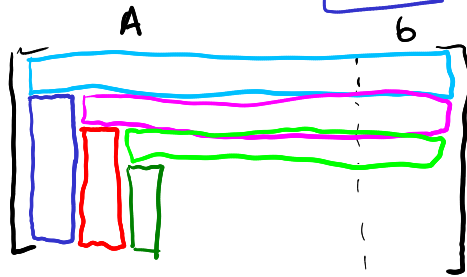
$$2x_1 + 3(1) + 4(2) = 19$$

$$2x_1 = 8$$

$$x_1 = 4$$

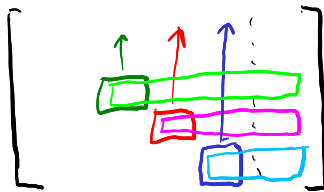
$$5x_2 + 12 = 17$$

$$x_2 = 1$$



Eliminate $\square$ with $\square$
Eliminate $\square$ with $\square$
etc

Back substitution



Solve for $\square$ with $\square$
Solve for $\square$ with $\square$
etc.

Example :

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 3 & 9 & -2 \\ 2 & 1 & -1 \end{bmatrix}$$

$$b = \begin{bmatrix} 1 \\ 9 \\ 2 \end{bmatrix}$$

$$x = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}$$

input : $n \times n$ matrix $[A]$, right-hand side $\{b\}$

output: solution $\{x\}$ of $[A]\{x\} = \{b\}$

// forward elimination

for $c \leftarrow 1$ to $n-1$ do

for $r \leftarrow c+1$ to n do

$f \leftarrow A[r, c] / A[c, c]$

$A[r, c:n] \leftarrow A[r, c:n] - f \cdot A[c, c:n]$

$b[r] \leftarrow b[r] - f \cdot b[c]$

end

end

// back substitution

for $r \leftarrow n, n-1, \dots, 1$ do

$s \leftarrow b[r]$

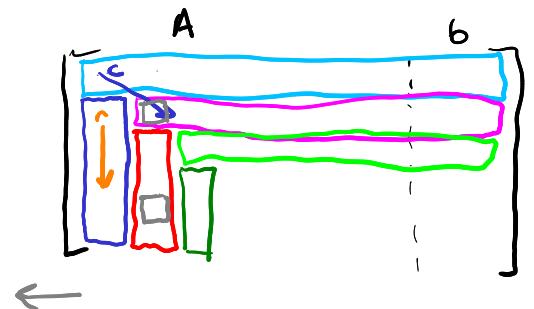
for $j \leftarrow r+1$ to n do

$s \leftarrow s - A[r, j] \cdot x[j]$

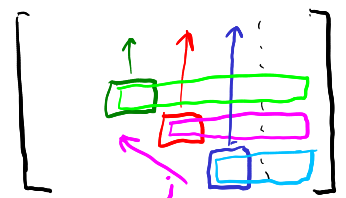
end

$x[r] \leftarrow s / A[r, r]$

end



Back substitution



Elimination
Backsub

$$A = \begin{bmatrix} 0 & 2 & 3 \\ 4 & 6 & 7 \\ 2 & -3 & 6 \end{bmatrix} \quad b = \begin{bmatrix} 8 \\ -3 \\ 5 \end{bmatrix}$$

$$f \leftarrow \underline{\text{Aug}[1,2]} / \underline{\text{Aug}[1,1]}$$

Divide by Zero

Solution: Partial Pivoting

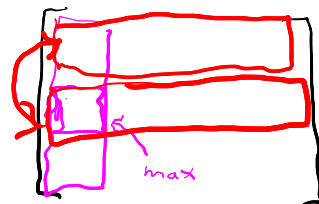
```

input :  $n \times n$  matrix  $[A]$ , right-hand side  $\{b\}$ 
output: solution  $\{x\}$  of  $[A]\{x\} = \{b\}$ 

// forward elimination
for  $c \leftarrow 1$  to  $n - 1$  do
    // find the largest entry in column  $c$ 
     $p \leftarrow \text{argmax}(\text{abs}(A[c:n, c])) + c - 1$ 
    // swap that row into the pivot position
     $A[c, :] \leftrightarrow A[p, :]$ 
     $b[c] \leftrightarrow b[p]$ 
    for  $r \leftarrow c + 1$  to  $n$  do
         $f \leftarrow A[r, c] / A[c, c]$ 
         $A[r, c:n] \leftarrow A[r, c:n] - f \cdot A[c, c:n]$ 
         $b[r] \leftarrow b[r] - f \cdot b[c]$ 
    end
end

// back substitution
for  $r \leftarrow n, n - 1, \dots, 1$  do
     $s \leftarrow b[r]$ 
    for  $j \leftarrow r + 1$  to  $n$  do
         $s \leftarrow s - A[r, j] \cdot x[j]$ 
    end
     $x[r] \leftarrow s / A[r, r]$ 
end

```



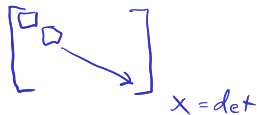
Algorithms and Operation Counting

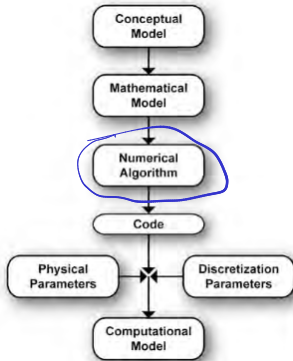
ASEN 3502 — Computational Methods

Please grab a handout in the back!

HW 1 - Due Thursday

Lab min through 1.3
 rec through 1.5





NM-11050-33

Today

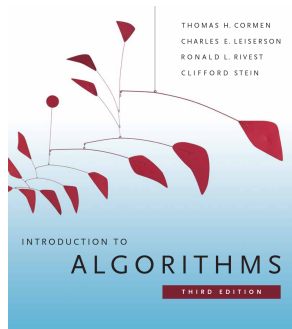
- Algorithms
- Operation Counting
- Asymptotic Notation

$O(n)$

What is an algorithm?

“Informally, an algorithm is any well-defined computational **procedure** that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**.”

— Cormen et al.



Introduction to Algorithms

Cormen, Leiserson,
Rivest, and Stein

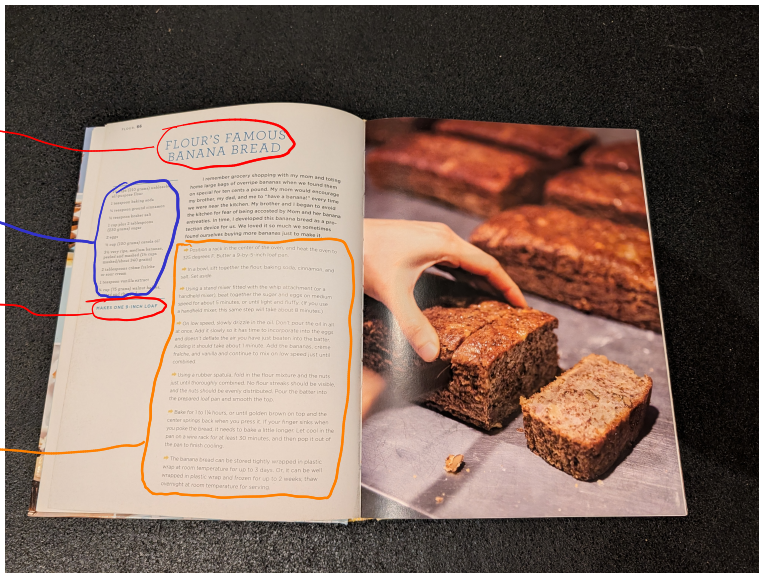
mitpress.mit.edu

Another type of algorithm

Outputs

Inputs

Procedure



Why algorithms?



- ▶ Algorithms are mathematical objects. Why are they useful?

Why algorithms?

- ▶ Algorithms are mathematical objects. Why are they useful?
 - ▶ We can reason about properties like correctness and resource use

Why algorithms?

- ▶ Algorithms are mathematical objects. Why are they useful?
 - ▶ We can reason about properties like correctness and resource use
 - “An algorithm is said to be **correct** if, for every input instance, it halts with the correct output.” — Cormen et al.

Why algorithms?

- ▶ Algorithms are mathematical objects. Why are they useful?
 - ▶ We can reason about properties like correctness and resource use

“An algorithm is said to be **correct** if, for every input instance, it halts with the correct output.” — Cormen et al.
 - ▶ We can make precise and rigorous statements without having to deal with issues like roundoff error, memory allocation, etc.

How do we represent algorithms?

Matrix-vector multiplication: three ways

English

To get entry r of $\{b\}$, multiply each entry in row r of $[A]$ by the matching entry of $\{x\}$, and add up the results. Do this for every row.

Code

```
b = zeros(n,1);  
for r = 1:n  
    for c = 1:n  
        b(r) = b(r) ...  
            + A(r,c)*x(c);  
    end  
end
```

Pseudocode

```
input      :  $n \times n$  matrix  $[A]$ ,  
              vector  $\{x\}$   
output    :  $\{b\} = [A]\{x\}$   
for  $r \leftarrow 1$  to  $n$  do  
     $b[r] \leftarrow 0$   
    for  $c \leftarrow 1$  to  $n$  do  
         $b[r] \leftarrow b[r] + A[r, c] \cdot x[c]$   
    end  
end
```

“What separates pseudocode from “real” code is that in pseudocode, we employ whatever expressive method is most clear and concise to specify a given algorithm.” — Cormen et al.

Analysis of algorithms

- ▶ *Analyzing* an algorithm means predicting the resources that an algorithm requires
 - ▶ For today, we will count floating point operations (“Flops”): multiplication, division, addition, and subtraction of floating point numbers

Analysis of algorithms

- ▶ *Analyzing* an algorithm means predicting the resources that an algorithm requires
 - ▶ For today, we will count floating point operations (“Flops”): multiplication, division, addition, and subtraction of floating point numbers
- ▶ **Discuss with your neighbor:** Consider the equation $[A]\{x\} = \{b\}$. As n gets larger, which requires more floating point operations? Finding $\{x\}$ given $[A]$ and $\{b\}$, or finding $\{b\}$ given $[A]$ and $\{x\}$?

$$\textcircled{1} \quad b = Ax$$

$$\textcircled{2} \quad Ax = b$$

Harder if using
Gaussian Elimination

Matrix-vector multiplication

input : $n \times n$ matrix $[A]$, vector $\{x\}$

output : $\{b\} = [A]\{x\}$

```
1 for r ← 1 to n do
2   b[r] ← 0
3   for c ← 1 to n do
4     b[r] ← b[r] + A[r, c] · x[c]
5   end
6 end
```

flops

0
0
2

times

$\sum_{r=1}^n 1 = n$
 $\sum_{r=1}^n \sum_{c=1}^n 1 = n^2$

$$\sum_{r=1}^n \sum_{c=1}^n 1 = \sum_{r=1}^n n = n^2$$

FLOPS: $2n^2$

Gaussian elimination

input : $n \times n$ matrix $[A]$, right-hand side $\{b\}$

output : solution $\{x\}$ of $[A]\{x\} = \{b\}$

// forward elimination

```

1 for c ← 1 to n-1 do
2   for r ← c+1 to n do
3     f ← A[r, c] / A[c, c]
4     for j ← c to n do
5       A[r, j] ← A[r, j] - f · A[c, j]
6     end
7     b[r] ← b[r] - f · b[c]
8   end
9 end

```

// back substitution

```

10 for r ← n, n-1, ..., 1 do
11   s ← b[r]
12   for j ← r+1 to n do
13     s ← s - A[r, j] · x[j]
14   end
15   x[r] ← s / A[r, r]
16 end

```

flops	times
0	$\sum_{i=1}^{n-1} 1 = n-1$
0	$\sum_{i=1}^{n-1} \frac{n^2-n}{2}$
2	$\sum_{i=1}^{n-1} \left\{ \frac{n^3}{3} + O(n^2) \right\}$
2	$\frac{n^2-n}{2}$
0	n
0	n
0	$\sum_{i=1}^{n-1} \frac{n^2-n}{2}$
2	n
1	n

Useful sum identities

$$\sum_{i=1}^m cf(i) = c \sum_{i=1}^m f(i) \quad \left| \quad \sum_{i=1}^m f(i) + g(i) = \sum_{i=1}^m f(i) + \sum_{i=1}^m g(i) \right.$$

$$\sum_{i=1}^m 1 = m$$

$$\rightarrow \sum_{i=1}^m i = \frac{m(m+1)}{2}$$

$$\sum_{i=k}^m 1 = m - k + 1$$

$$\sum_{i=k}^m i^2 = \frac{m(m+1)(2m+1)}{6} - \frac{(k-1)k(2k-1)}{6} = \frac{m^3}{3} + O(m^2)$$

$$\sum_{c=1}^{n-1} \sum_{r=c+1}^n 1 = \sum_{c=1}^{n-1} (n-c) = \sum_{c=1}^{n-1} n - \sum_{c=1}^{n-1} c$$

$$= n(n-1) - \frac{(n-1)n}{2} = \frac{n^2-n}{2}$$

$$\sum_{c=1}^{n-1} \sum_{r=c+1}^n \sum_{j=c}^n 1 = \frac{n^3}{3} - \frac{n}{3}$$

see flop-count.pdf section 2.2

<https://asen3502.com/lectures/020-algorithms/flop-count.pdf>

$$\text{FLOPS: } \frac{n^2-n}{2} + 2\left(\frac{n^3}{3} + O(n^2)\right) + 2\frac{n^2-n}{2}$$

$$\text{FLOPS: } 2\frac{n^3}{3} + O(n^2) + 2\frac{n^2-n}{2} + n$$

function $[C] = \text{matmult}([A], [B])$	flops	times
for $i = 1$ to n	O	n
for $j = 1$ to n	O	n^2
$c_{ij} = 0$	O	$\vdots$
for $k = 1$ to n	O	n^3
$c_{ij} = c_{ij} + \underline{a_{ik}} \underline{b_{kj}}$	2	n^3
end		
end	,	
end		
return $[C]$		
end		

Asymptotic notation

- ▶ Asymptotic notation is a tool for keeping track of only the terms that are most important as a function approaches an asymptote

Asymptotic notation

- ▶ Asymptotic notation is a tool for keeping track of only the terms that are most important as a function approaches an asymptote
- ▶ Chapra: $O(m^n)$ means “terms of order m^n and lower”

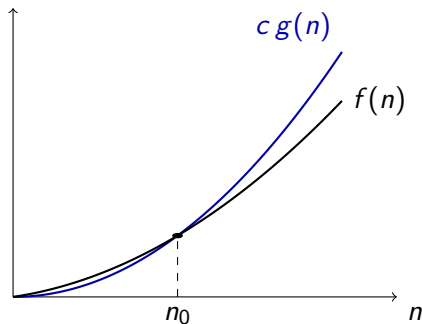
Asymptotic notation

- ▶ Asymptotic notation is a tool for keeping track of only the terms that are most important as a function approaches an asymptote
- ▶ Chapra: $O(m^n)$ means “terms of order m^n and lower”

Definition

$f(n) = O(g(n))$ as $n \rightarrow \infty$ if there exist positive constants c and n_0 such that

$$0 \leq f(n) \leq c g(n) \quad \text{for all } n \geq n_0$$



$$f(n) \leq c g(n) \quad \text{for all } n \geq n_0$$

← $f(n) = 3n^2 + 5n + 2$

$$f(n) = 3n^2 + 5n + 2$$

$$c = 4.0$$

$$n_0 = 6$$

then $0 \leq f(n) \leq c g(n)$
for all $n \geq n_0$

Therefore $f(n) = O(n^2)$

$c \geq \text{sum of coefficients}$

$$n_0 = 1$$

$$c = 10$$

$$n_0 = 1$$

Useful Rules

1) $c f(n) = O(f(n)) \quad \forall c > 0$

2) if $f_1(n) = O(n^a)$ and $f_2(n) = O(n^b)$, $f_1(n) + f_2(n) = O(n^{\max(a,b)})$

3) if $f_1(n) = O(n^a)$ and $f_2(n) = O(n^b)$, $f_1(n) \cdot f_2(n) = O(n^{a+b})$

4) if $|f(n)|$ is bounded, $f(n) = O(1)$

Example: $f(n) = \underbrace{n^2}_{O(n^2)} + \underbrace{12n}_{O(n)} \underbrace{\sin(n)}_{O(1)} + \underbrace{50}_{O(1)} = O(n^2)$



Gaussian elimination

input : $n \times n$ matrix $[A]$, right-hand side $\{b\}$

output : solution $\{x\}$ of $[A]\{x\} = \{b\}$

// forward elimination

```
for c ← 1 to n-1 do ←  $\leq n$ 
  for r ← c+1 to n do ←  $\leq n$ 
    f ←  $A[r, c] / A[c, c]$ 
    for j ← c to n do ←  $\leq n$ 
      |  $A[r, j] \leftarrow A[r, j] - f \cdot A[c, j]$ 
    end
     $b[r] \leftarrow b[r] - f \cdot b[c]$ 
  end
end
```

end

// back substitution

```
for r ← n, n-1, ..., 1 do
  s ←  $b[r]$ 
  for j ← r+1 to n do
    |  $s \leftarrow s - A[r, j] \cdot x[j]$ 
  end
   $x[r] \leftarrow s / A[r, r]$ 
end
```

end

flops	Order of times called
x	} $O(n)$
x	
✓	} $O(n^2)$
x	
✓	} $O(n^3)$
x	
✓	$O(n^2)$
x	} $O(n)$
x	
x	
✓	} $O(n^2)$
✓	
✓	$O(n)$

$O(n^3)$ Flops

Today

LU-Factorization

HW1 Due tomorrow

HW2 Assigned tonight

Lab 1: catch up with team to

1.3 (minimum)

1.5 (recommended)

before Friday

Study Hall tonight!

Recall: with a triangular matrix, T , solving $Tx=b$ for x is $O(n^2)$

Suppose we have

$LU = A$
 $\uparrow \quad \uparrow$
 lower triangular upper triangular

$$Ax = b$$

$$L(Ux) = b$$

$$\underbrace{L}_{d} Ux = b$$

0. factorize A into LU

1. solve $Ld = b$

2. solve $Ux = d$

both steps solve with a triangular matrix

$$\begin{aligned} &\Rightarrow L \quad \Rightarrow U \\ &\left(\begin{bmatrix} I \\ \begin{bmatrix} 0 & 0 & 0 \\ 2 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{bmatrix} \right) \begin{bmatrix} 2 & 3 & 4 \\ 0 & 5 & 6 \\ 2 & 8 & 17 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 4 \\ 4 & 11 & 14 \\ 2 & 8 & 17 \end{bmatrix} \\ &\left(I + \begin{bmatrix} 0 & 0 & 0 \\ 2 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \right) \begin{bmatrix} 2 & 3 & 4 \\ 0 & 5 & 6 \\ 0 & 5 & 13 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 4 \\ 4 & 11 & 14 \\ 0 & 5 & 13 \end{bmatrix} \\ &\left(I + \begin{bmatrix} 0 & 0 & 0 \\ 2 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix} \right) \begin{bmatrix} 2 & 3 & 4 \\ 0 & 5 & 6 \\ 0 & 0 & 7 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 4 \\ 4 & 11 & 14 \\ 0 & 0 & 7 \end{bmatrix} \\ &\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 3 & 4 \\ 0 & 5 & 6 \\ 0 & 0 & 7 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 4 \\ 4 & 11 & 14 \\ 2 & 8 & 17 \end{bmatrix} \\ &\quad \quad \quad L \quad \quad \quad U \end{aligned}$$

$$L = \begin{bmatrix} 1 & & & \\ f_{21} & 1 & & \\ f_{31} & f_{32} & \ddots & \\ \vdots & \vdots & \ddots & \ddots \\ f_{n1} & & & 1 \end{bmatrix}$$

$U =$ result of GE

f_{ij} = factor from GE for eliminating element ij

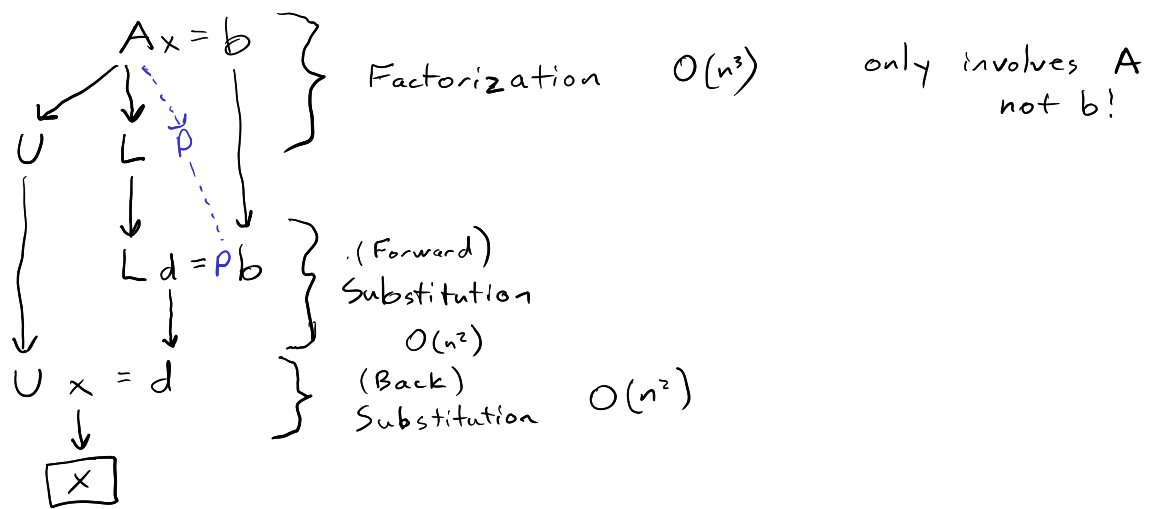
LU factorization: run GE and save factors in L

$$A = \begin{bmatrix} 2 & 1 \\ 6 & 5 \end{bmatrix} \begin{matrix} \textcircled{1} \\ \textcircled{2} \end{matrix} \text{ subtract } \underline{3 \times \textcircled{1}} \text{ from } 2$$

$$L = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}$$

$$LU = \begin{bmatrix} 2 & 1 \\ 6 & 5 \end{bmatrix} \stackrel{\checkmark}{=} A$$



If we have many b 's to solve for, factorize once ($O(n^3)$) then do forward and back substitution for each b ($O(n^2)$)!

What about Pivoting?

$$LU = PA$$

↑ permutation matrix (keep track of pivots)

$$\begin{bmatrix} 4 & 11 & 14 \\ 2 & 3 & 4 \\ 2 & 8 & 17 \end{bmatrix}_{A'} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}_P \begin{bmatrix} 2 & 3 & 4 \\ 4 & 11 & 14 \\ 2 & 8 & 17 \end{bmatrix}_A$$

(Red arrow indicates row 1 of A moves to row 2 of A')

each row i determines which row j of A ends up in A'
 $p_{ij} = 1$ if row j of A becomes row i of A'

Exercise 2

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ c_1 & c_2 & c_3 \\ b_1 & b_2 & b_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix}$$

$$\begin{aligned} Ax &= b \\ LU &= PA \\ \rightarrow PAx &= Pb \\ LUx &= P| \end{aligned}$$

$$\begin{bmatrix} -c- \\ -b- \\ -a- \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix}$$

Matrix Inverse Condition Number Roundoff Error

Homework 2 Due Thursday

$$\begin{aligned} Ax &= b \\ \text{inverse} \rightarrow A^{-1}b &= x \\ AA^{-1}b &= Ax = b \\ AA^{-1}b &= b \\ AA^{-1} &= I \\ &\quad (3 \times 3) \end{aligned}$$

$$A \begin{bmatrix} a_{11}^{-1} \\ a_{21}^{-1} \\ a_{31}^{-1} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad A \begin{bmatrix} a_{12}^{-1} \\ a_{22}^{-1} \\ a_{32}^{-1} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad A \begin{bmatrix} a_{13}^{-1} \\ a_{23}^{-1} \\ a_{33}^{-1} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Matrix Inverse via LU factorization

input : $n \times n$ matrix $[A]$

output: $[A]^{-1}$

$[L], [U] \leftarrow \text{lu_decompose}([A])$

for $k \leftarrow 1$ to n do

$\{e_k\} \leftarrow$ unit vector with 1 at element k ; 0 elsewhere
 $A^{-1}[:, k] \leftarrow \text{lu_solve}([L], [U], \{e_k\})$

end

What can go wrong?

from APPM 2360

$Ax=b$
does not
have a unique
solution

$\Leftrightarrow A$ is singular $\Leftrightarrow \det(A) = 0 \Leftrightarrow A$ is not full rank $\Leftrightarrow A^{-1}$ does not exist

$$\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$$

not full rank

What if A is almost singular?

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

large cond $\Rightarrow$ nearly singular

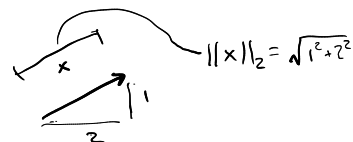
$\| \|$ means "norm"

Book covers many types of norms

For now $\| \| = \| \|_2$ "2-norm"

For a vector x , $\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$, "length" of x

For a matrix A , $\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \sqrt{\lambda_{\max}}$
"The most A can elongate a vector" $\nwarrow$ max eigenvalue



Key property of cond:

$$\frac{\|\Delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\Delta A\|}{\|A\|}$$

relative error
in result

relative error in
matrix

Roundoff Errors

Computer uses only 3 significant figures

747 takeoff mass

3.97×10^5 kg

burns 4.21 kgs fuel

$$\begin{array}{r} 397,000 \text{ kg} \\ - 4.21 \text{ kg} \\ \hline 396,995.71 \end{array} \rightarrow 3 \text{ sig figs} \rightarrow 3.97 \times 10^5 \text{ kg}$$

How computers store numbers: bits (1 or 0)

Humans think in decimal digits
0, ..., 9

Unsigned
Integers

3

137

43

1000	100	10	1
0	0	0	3
0	1	3	7
0	0	4	3

Scientific
Notation

$$\pm s \times 10^e$$

↑ significand ↗ exponent

Computers think in Binary
0, 1

1 byte

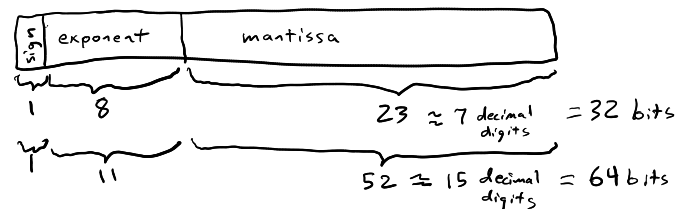
128	64	32	16	8	4	2	1
0	0	0	0	0	0	1	1
1	0	0	0	1	0	0	1
0	0	1	0	1	0	1	1

Floating Point
IEEE 754

$$\pm (1+f) \times 2^{(e-127)}$$

↑ mantissa ↗ exponent

precision
single
double



matlab/julia $\text{eps}(x)$ distance to next floating point number above x

$$\frac{\|\Delta A\|}{\|A\|} \approx \text{eps}(1.0)$$

$$\frac{\|\Delta x\|}{\|x\|} \leq \text{cond}(A) \epsilon$$

Please Grab a handout in the back.

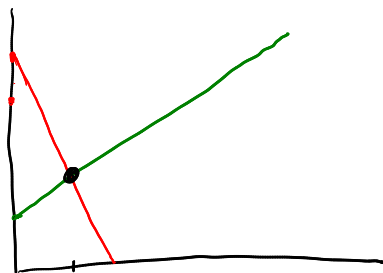
Today

Solving Nonlinear Function
Bracketing Methods: Most important: Bisection

Announcements / Reminders

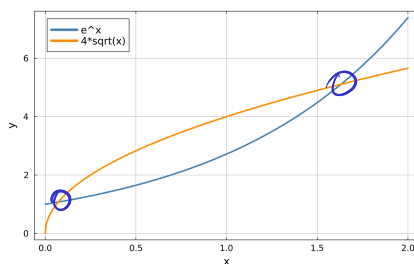
Lab 1 due Thursday - Check Auto-graded!
HW 1 Released (Graded)

Linear



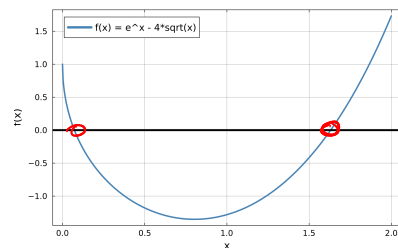
$$\begin{aligned} -2x + 4 &= x + 1 \\ x_2 &= -2x_1 + 4 & x_2 &= x_1 + 1 \\ \boxed{x &= 1} \end{aligned}$$

Nonlinear



$$\begin{aligned} e^x &= 4\sqrt{x} \\ \text{no analytical in} \\ \text{"simple functions"} \end{aligned}$$

⇒



$$\begin{aligned} f(x) &= 0 = e^x - 4\sqrt{x} \\ \text{find "roots"} \\ \text{x-values where } f(x) &= 0 \end{aligned}$$

Bracketing

Open

↑ Today

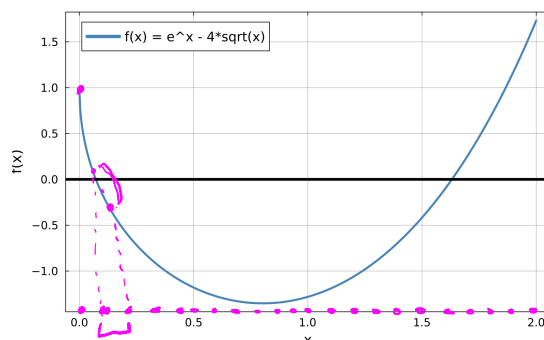
Incremental Search

Incremental Search

input : function f , search range $[x_{\min}, x_{\max}]$, number of steps n
output: list of brackets $[x_l, x_u]$, each containing a root

```

 $\Delta x \leftarrow (x_{\max} - x_{\min}) / n$ 
brackets  $\leftarrow \{\}$ 
 $x_l \leftarrow x_{\min}$ 
for  $k \leftarrow 1$  to  $n$  do
     $x_u \leftarrow x_l + \Delta x$ 
    if  $f(x_l) \cdot f(x_u) < 0$  then
        // sign change ⇒ a root lies in  $[x_l, x_u]$ 
        append  $[x_l, x_u]$  to brackets
    end
     $x_l \leftarrow x_u$ 
end
return brackets
    
```



↑ if $f(x)$ is continuous and $f(x_l)$ and $f(x_r)$ have different signs, a root is between x_l and x_r

Strength: Can find multiple roots

Weakness: Can miss roots, slow at finding one root
(we will see shortly)

error $\rightarrow E_a = \frac{\Delta x}{n-1}$

Bisection

Bisection

input : function f , bracket $[x_l, x_u]$ with $f(x_l) \cdot f(x_u) < 0$, tolerance ε_s
output: estimate x_r of the root

```

 $x_r \leftarrow x_l$ 
repeat
     $x_{r,old} \leftarrow x_r$ 
     $x_r \leftarrow (x_l + x_u) / 2$ 
    // keep the half that still brackets the root
    if  $f(x_l) \cdot f(x_r) < 0$  then
         $x_u \leftarrow x_r$ 
    else if  $f(x_l) \cdot f(x_r) > 0$  then
         $x_l \leftarrow x_r$ 
    else
        return  $x_r$  // exact root
    end
until  $\varepsilon_a < \varepsilon_s$ 
return  $x_r$ 
    
```

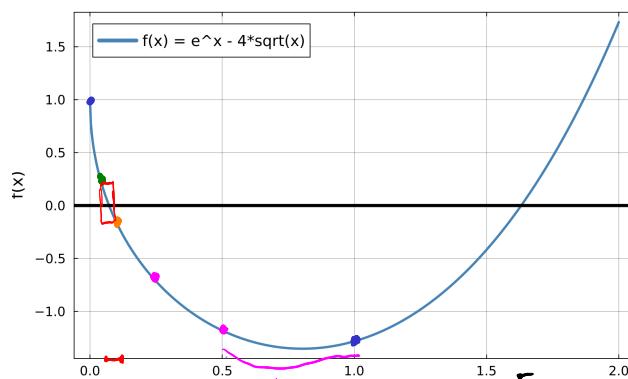
Iter: 1

2

3

4

5



0.0675 0.125

E_a

$\Delta x = 1$

$\Delta x = 0.5$

$\Delta x = 0.25$

$\Delta x = 0.125$

$\Delta x = 0.0675$

$E_a = \frac{\Delta x}{2^n}$

$$E_a = \frac{\Delta x}{2^n}$$

function calls
for desired $\frac{\Delta x}{E_s}$

Incremental

$$E_a = E_s = \frac{\Delta x}{n-1}$$

$$n-1 = \frac{\Delta x}{E_s}$$

$$n = \frac{\Delta x}{E_s} + 1$$

$$O\left(\frac{\Delta x}{E_s}\right)$$

Bisection

$$E_a = E_s = \frac{\Delta x}{2^n}$$

$$2^n = \frac{\Delta x}{E_s}$$

$$n = \log_2\left(\frac{\Delta x}{E_s}\right)$$

$$O\left(\log\left(\frac{\Delta x}{E_s}\right)\right)$$

$$E_s = 0.01 \Delta x$$

$$\frac{\Delta x}{E_s} = 100$$

101 func
evals

$$\log(100) = 6.64$$

7 function
evals

$$E_s = 0.0001 \Delta x$$

$$\frac{\Delta x}{E_s} = 10,000$$

10,001 func
evals

14 func
evals

False Position

False Position

input : function f , bracket $[x_l, x_u]$ with $f(x_l) \cdot f(x_u) < 0$,
tolerance ε_s

output: estimate x_r of the root

$x_r \leftarrow x_l$

repeat

$x_{r,old} \leftarrow x_r$

$$x_r \leftarrow x_u - \frac{f(x_u)(x_l - x_u)}{f(x_u) - f(x_l)}$$

// keep the side that still brackets the root

if $f(x_l) \cdot f(x_r) < 0$ then

$x_u \leftarrow x_r$

else if $f(x_l) \cdot f(x_r) > 0$ then

$x_l \leftarrow x_r$

else

 return x_r

end

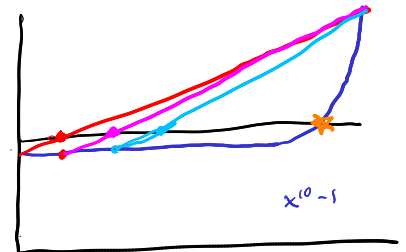
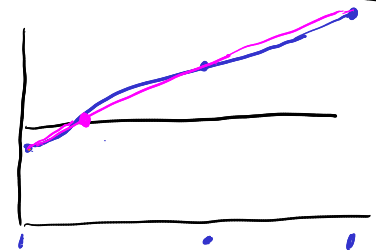
$$\varepsilon_a \leftarrow \left| \frac{x_r - x_{r,old}}{x_r} \right| \cdot 100\%$$

until $\varepsilon_a < \varepsilon_s$

return x_r

← only difference
from bisection

Problem: Doesn't
handle
curvature well



Today

Open Methods: Newton-Raphson + Friends

Secant, Brent's, Fixed Point Iter
(maybe)

Reminders

Lab 1 due tomorrow

First: Errors (Chapra's notation)

E : absolute error (units)

ϵ : relative (percent) error

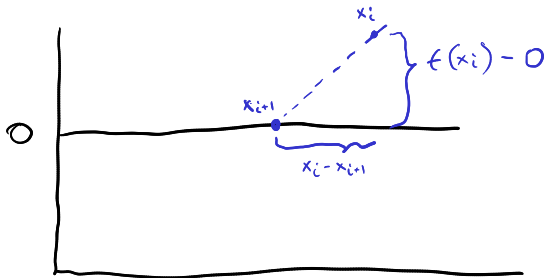
Subscripts

t : true (typically unknown)

a : approximate

s : stopping

Newton-Raphson: use $f'(x)$



$$f'(x_i) = \frac{f(x_i) - 0}{x_i - x_{i+1}} \quad \text{rise over run}$$

$$(x_i - x_{i+1}) f'(x_i) = f(x_i)$$

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Newton-Raphson

input: function f and its derivative f' , initial guess x_0 , tolerance ϵ_s , maximum iterations N

output: estimate x_r of the root

$x_r \leftarrow x_0$

for $i \leftarrow 1$ to N do

$x_{r,old} \leftarrow x_r$

$x_r \leftarrow x_{r,old} - \frac{f(x_{r,old})}{f'(x_{r,old})}$ ← update

$\epsilon_a \leftarrow \left| \frac{x_r - x_{r,old}}{x_r} \right| \cdot 100\%$

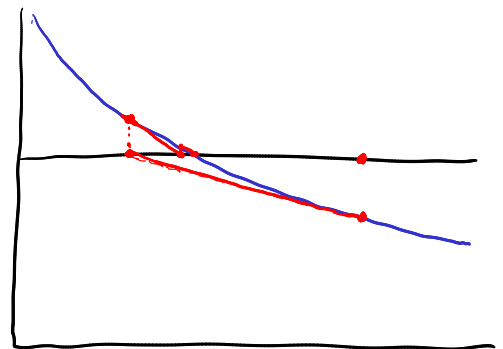
if $\epsilon_a < \epsilon_s$ then

 return x_r

end

end

return x_r // did not converge in N iterations



Beauty of Newton-Raphson: Extremely fast convergence near root!

Bisection

$$E_{a,i+1} = \frac{1}{2} E_{a,i}$$

"linear convergence"

number of sig figs increases linearly with i

(different terminology from $E_{a,i} = O((\frac{1}{2})^k)$)

Guaranteed to converge if

 - Δx contains root

 - $f(x)$ is continuous

fast

Newton Raphson

Near Root

$$\rightarrow E_{t,i+1} \approx \frac{-f''(x_r)}{2f'(x_r)} E_{t,i}^2 \quad \text{"can be shown"}$$

"quadratic convergence"

number of sigfigs doubles every iteration

Can diverge

FAST near root

Downside: Newton requires $f'(x)$

Secant Method

$$f'(x_i) \approx \frac{f(x_{i-1}) - f(x_i)}{x_{i-1} - x_i}$$

plug into Newton update

$$x_{i+1} = x_i - \frac{f(x_i)(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)}$$

Secant Method

input : function f , initial guesses x_{-1} and x_0 , tolerance ε_s , maximum iterations N

output: estimate x_r of the root

$x_{\text{prev}} \leftarrow x_{-1}; \quad x_r \leftarrow x_0$

for $i \leftarrow 1$ **to** N **do**

$x_{r,\text{old}} \leftarrow x_r$

$x_r \leftarrow x_{r,\text{old}} - \frac{f(x_{r,\text{old}})(x_{\text{prev}} - x_{r,\text{old}})}{f(x_{\text{prev}}) - f(x_{r,\text{old}})}$

$x_{\text{prev}} \leftarrow x_{r,\text{old}}$

$\varepsilon_a \leftarrow \left| \frac{x_r - x_{r,\text{old}}}{x_r} \right| \cdot 100\%$

if $\varepsilon_a < \varepsilon_s$ **then**

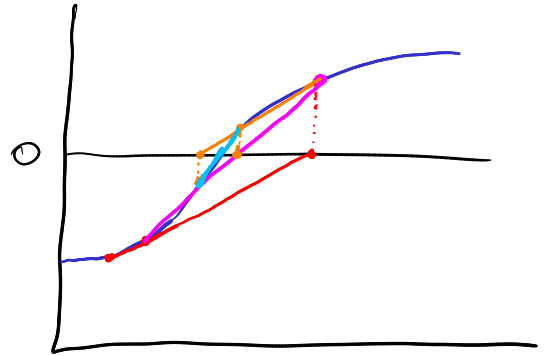
return x_r

end

end

return x_r // did not converge in N iterations

only difference from Newton



Brent's Method

Brent's Method

input : function f , bracket $[a, b]$ with $f(a) \cdot f(b) < 0$, tolerance ε_s

output: estimate b of the root

// b is the best estimate, c is on the other side of the root, a is the previous b

$c \leftarrow a$

// d is the signed step b will move by, e is the previous step;
both start as the bracket width

$d \leftarrow b - a$; $e \leftarrow d$

repeat

if $f(b) \cdot f(c) > 0$ then

 // b and c no longer bracket the root; a does

$c \leftarrow a$; $d \leftarrow b - a$; $e \leftarrow d$

end

if $|f(c)| < |f(b)|$ then

 // make b the point with the smallest $|f|$

$a \leftarrow b$; $b \leftarrow c$; $c \leftarrow a$

end

$m \leftarrow (c - b) / 2$

// bisection step

$\delta \leftarrow \varepsilon_s |b|$

// absolute tolerance

if $|m| \leq \delta$ or $f(b) = 0$ then

 return b

end

if $|e| < \delta$ or $|f(a)| \leq |f(b)|$ then

$d \leftarrow m$; $e \leftarrow m$

// bisection

else

$s \leftarrow f(b) / f(a)$

 if $a = c$ then

 // only two distinct points: secant step

$p \leftarrow 2ms$; $q \leftarrow 1 - s$

 else

 // three distinct points: inverse quadratic

 interpolation

$q \leftarrow f(a) / f(c)$; $r \leftarrow f(b) / f(c)$

$p \leftarrow s(2mq(q-r) - (b-a)(r-1))$

$q \leftarrow (q-1)(r-1)(s-1)$

 end

 if $p > 0$ then $q \leftarrow -q$

 else $p \leftarrow -p$

 if $2p < 3mq - |\delta q|$ and $p < |eq/2|$ then

 // interpolated step stays in the bracket and shrinks

 it fast enough

$e \leftarrow d$; $d \leftarrow p/q$

 else

$d \leftarrow m$; $e \leftarrow m$

// bisection

 end

end

$a \leftarrow b$

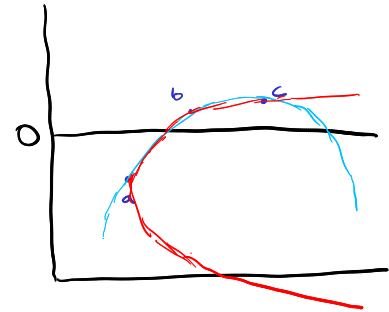
- Combination of Bracketing and Open Methods
- Does not require derivatives
- Basis for Matlab's `fzero`

- Guaranteed to converge
- As fast near root as open methods

← bisection

← secant

← inverse quadratic



Fixed Point

Previously $f(x) = 0$

Fixed Point

$$x = g(x)$$

repeatedly evaluate $g(x)$

Today

Systems of nonlinear equations

Newton-Raphson for systems

Numerical differentiation

Announcements

- HW 3 due Thurs

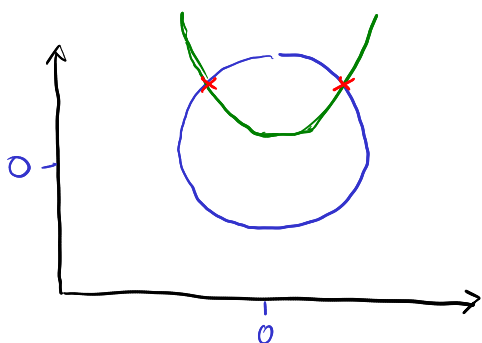
- Exam Next Monday

- Will release Eqn. Sheet / practice exam today / tomorrow

- Calculator

- Study HW, Labs, Notes

Up to now $f(x) = 0$



$$x_1^2 + x_2^2 = 4$$

$$x_2 = x_1^2$$

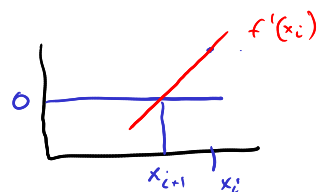
$$\{f\}(\{x\}) = \{0\}$$

$$\vec{f}(\vec{x}) = \vec{0}$$

$$\vec{f}(\vec{x}) = \begin{bmatrix} x_1^2 + x_2^2 - 4 \\ x_2 - x_1^2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Newton-Raphson for scalar:

$$f(x_i) = f(x_{i+1}) + f'(x_i)(x_i - x_{i+1})$$



vector case:

first element of f →

$$f_1(\vec{x}_i) = f_1(\vec{x}_{i+1}) + \frac{\partial f_1}{\partial x_1}(x_{1,i} - x_{1,i+1}) + \frac{\partial f_1}{\partial x_2}(x_{2,i} - x_{2,i+1}) + \dots + \frac{\partial f_1}{\partial x_n}(x_{n,i} - x_{n,i+1})$$

choose $\vec{x}_{i+1}$ so this is zero

$$= \sum_{j=1}^n \frac{\partial f_1}{\partial x_j}(x_{j,i} - x_{j,i+1})$$

$$f_2(\vec{x}_i) = \sum_{j=1}^n \frac{\partial f_2}{\partial x_j}(x_{j,i} - x_{j,i+1})$$

$$\vdots$$

$$f_n(\vec{x}_i) = \sum_{j=1}^n \frac{\partial f_n}{\partial x_j}(x_{j,i} - x_{j,i+1})$$

$$\vec{f}(\vec{x}_i) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial f_n}{\partial x_1} & \dots & \frac{\partial f_n}{\partial x_n} \end{bmatrix} \left(\begin{bmatrix} x_{1,i} \\ \vdots \\ x_{n,i} \end{bmatrix} - \begin{bmatrix} x_{1,i+1} \\ \vdots \\ x_{n,i+1} \end{bmatrix} \right)$$

$$\underbrace{\vec{f}(\vec{x}_i)}_{\text{known}} = \underbrace{J}_{\text{known}} (\underbrace{\vec{x}_i}_{\text{known}} - \underbrace{\vec{x}_{i+1}}_{\text{unknown}})$$

$$J^{-1} \vec{f}(\vec{x}_i) = \vec{x}_i - \vec{x}_{i+1}$$

$$\boxed{\vec{x}_{i+1} = \vec{x}_i - J^{-1} \vec{f}(\vec{x}_i)}$$

(scalar)

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

want $\vec{x}_{i+1} = \vec{x}_i + \Delta \vec{x}$

$$\Delta \vec{x} = \vec{x}_{i+1} - \vec{x}_i$$

solve $J \Delta \vec{x} = -\vec{f}(\vec{x}_i)$

$$\vec{x}_{i+1} \leftarrow \vec{x}_i + \Delta \vec{x}$$

or

$$\vec{x}_{i+1} \leftarrow \vec{x}_i - J \backslash \vec{f}(\vec{x}_i)$$

$$A \backslash b \equiv \text{solution to } Ax = b$$

Newton-Raphson for Systems

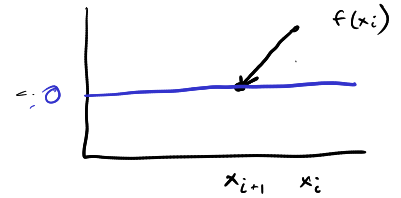
input : function $\{f\} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ and its Jacobian $[J]$ (with $J_{ij} = \partial f_i / \partial x_j$), initial guess $\{x\}_0$, tolerance ε_s , maximum iterations N

output: estimate $\{x\}_r$ of the root

```

{x}_r ← {x}_0
for i ← 1 to N do
    {x}_{r,old} ← {x}_r
    solve [J]({x}_{r,old}) {Δx} = -{f}({x}_{r,old})
    {x}_r ← {x}_{r,old} + {Δx}
    ε_a ← (||{x}_r - {x}_{r,old}|| / ||{x}_r||) * 100%
    if ε_a < ε_s then
        return {x}_r
    end
end
return {x}_r // did not converge in N iterations
    
```

(scalar)



Two Challenges

1. Convergence

2. Calculating Jacobian

Optimization

$$\text{minimize } \sum_{k=1}^n (f_k(\vec{x}))^2$$

Strength

"quadratic convergence" near root

Calculating Derivatives

Symbolic

- By hand
- Computer Algebra System (CAS)
 - e.g. Symbolic Toolbox
- Automatic Code gen

Numerical

Automatic

- Tracks exact derivative through each operation

Numerical Differentiation

Forward: $f'(x) \approx \frac{f(x+h) - f(x)}{h}$

Central: $f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$ ← more accurate

Column of Jacobian → $\frac{\partial \vec{f}}{\partial x_j} \approx \frac{\vec{f}(\vec{x} + h\vec{e}_j) - \vec{f}(\vec{x})}{h}$

size of h ?

Numerical Jacobian (forward difference)

input : function $\{f\} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, point $\{x\}$, step size h

output: approximation $[J]$ of the Jacobian at $\{x\}$, $J_{ij} \approx \partial f_i / \partial x_j$

$\{f\}_0 \leftarrow \{f(\{x\})\}$

for $j \leftarrow 1$ **to** n **do**

$\{e_j\} \leftarrow$ unit vector in the j direction // j th column of $[I]$

$[J][:, j] \leftarrow \frac{\{f(\{x\} + h\{e_j\})\} - \{f\}_0}{h}$ // j th column of $[J]$

end

return $[J]$

$$h \approx \sqrt{\epsilon} \times$$

↑
machine precision ϵ

Today

Logarithmic + Exponential Growth

Bisection requires $\log_2\left(\frac{\Delta x}{\epsilon_s}\right)$ function calls
 $\uparrow$
 usually c flops

Reminders

- HW 3 Due Tomorrow
- Exam 1 Monday
 - Calculator, pencil
 - Be familiar w/ Eqn sheet (I will print)

Algorithm 1

Pick up end of strip
 for $i = 1$ to L/ϵ_s
 Fold one section (accordion style)
 end

time	times executed
t_p	1
t_f	$\lceil L/\epsilon_s \rceil$

runtime
 $t_p + t_f \lceil L/\epsilon_s \rceil$
 $t_p + t_f \frac{L}{\epsilon_s} = O(L)$

Algorithm 2

1 Pick up end of strip
 2 $l_t \leftarrow L$
 3 while $l_t > \epsilon_s$
 4 ~~Run to other end~~
 5 Pick up end and fold in half
 6 $l_t \leftarrow l_t/2$
 7 end

time	times executed
t_p	1
0	1
t_f	$\log_2\left(\frac{L}{\epsilon_s}\right)$
0	

Iter	l_t
0	L
1	$\frac{L}{2}$
2	$\frac{L}{4}$
k	$\frac{L}{2^k}$

$\epsilon_s = l_t = \frac{L}{2^k}$
 $2^k = \frac{L}{\epsilon_s}$
 $k = \log_2\left(\frac{L}{\epsilon_s}\right)$

runtime:
 $t_p + t_f \log_2\left(\frac{L}{\epsilon_s}\right)$
 $= O(\log(L))$

Algorithm 2 (Running)

1 Pick up end of strip
 2 $l_t \leftarrow L$
 3 while $l_t > \epsilon_s$
 4 Run to other end
 5 Pick up end
 6 $l_t \leftarrow l_t/2$
 7 end

time	times executed
t_p	1
0	
l_t/v	$\log\left(\frac{L}{\epsilon_s}\right)$
t_p	$\log\left(\frac{L}{\epsilon_s}\right)$

runtime:
 $t_p + \sum_{j=1}^{\log_2(L/\epsilon_s)} \left(\frac{L}{2^j} \frac{1}{v} + t_p \right)$
 $= t_p \left(1 + \log_2\left(\frac{L}{\epsilon_s}\right) \right) + \sum_{j=1}^{\log_2(L/\epsilon_s)} \frac{L}{2^j} \frac{1}{v}$
 $\leq t_p \left(1 + \log_2\left(\frac{L}{\epsilon_s}\right) \right) + \frac{L}{v} \sum_{j=1}^{\infty} 2^{-j}$
 $= t_p \left(1 + \log_2\left(\frac{L}{\epsilon_s}\right) \right) + \frac{2L}{v}$
 $= O(L)$

$v = 10m/s$

Alg 1 and Alg 2(R) both have $O(L)$ runtime
 $\downarrow$ $\downarrow$
 $\frac{t_f}{\epsilon_s} L$ $\frac{2}{v} L$

$\frac{\epsilon_s}{t_f} = \frac{1/3 m}{1 s}$
 $\frac{t_f}{\epsilon_s} = 3$

$\frac{2}{v} = 0.2$

Alg 2 (R)
 takes $\sim 15 \times$ Alg

Logarithmic Growth

$O(1)$ $O(\log(n))$ $O(n^{\epsilon})$ $O(n^{1/2})$ $O(n)$ $O(n^2)$ $O(n^3)$
 $\log(n)$ grows slower asymptotically than any polynomial n^ϵ $\epsilon > 0$ $\nearrow O(n^\epsilon)$

$$\begin{matrix} f(n) = O(n) \\ \uparrow \\ f(n) \in O(n) \end{matrix}$$

$$O(\log(n)) \subsetneq O(n^\epsilon) \quad \forall \epsilon > 0$$

↑
strict subset

For sets of functions $O(f(n))$ and $O(g(n))$: $O(f(n)) \subsetneq O(g(n))$ means

1. $O(g(n))$ contains every function in $O(f(n))$

2. $O(g(n))$ contains at least 1 function **NOT** in $O(f(n))$

How can we prove this

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow O(f(n)) \subsetneq O(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{\log(n)}{n^\epsilon} \stackrel{\text{L'Hopital}}{=} \lim_{n \rightarrow \infty} \frac{n^{-1}}{\epsilon n^{\epsilon-1}} = \lim_{n \rightarrow \infty} \frac{1}{\epsilon n^\epsilon} = 0$$

$$\therefore O(\log(n)) \subsetneq O(n^\epsilon) \quad \forall \epsilon > 0$$

$$\log_a(n) = \frac{\log_b(n)}{\log_b(a)} \quad \leftarrow \text{constant}$$

$$\therefore O(\log_a(n)) = O(\log_b(n)) \quad \forall a, b > 0$$

Exponential Growth

$a^n, a > 1$ grows faster than any polynomial

$$e^n = \sum_{j=0}^{\infty} \frac{n^j}{j!} \quad a^n = \sum_{j=0}^{\infty} \frac{(\log(a))^j n^j}{j!} \geq \frac{(\log(a))^{k+1} n^{k+1}}{(k+1)!}$$

$$\lim_{n \rightarrow \infty} \frac{n^k}{a^n} \leq \lim_{n \rightarrow \infty} \frac{(k+1)!}{(\log(a))^{k+1}} \cdot \frac{n^k}{n^{k+1}} = \lim_{n \rightarrow \infty} \frac{1}{n} = 0$$

$$\therefore O(n^k) \subsetneq O(a^n) \quad \forall a > 1, k > 0$$

Let $1 < a < b$

$$O(a^n) \subsetneq O(b^n) \quad \forall 1 < a < b$$

Factorial Growth

$n!$
 $= n(n-1)(n-2)\dots 1$ grows faster than any exponential

$$\lim_{n \rightarrow \infty} \frac{a^n}{n!} = \lim_{n \rightarrow \infty} \frac{\overbrace{a \cdot a \cdot a \dots a}^N \cdot \overbrace{a \cdot a \dots a}^{n-N}}{\underbrace{1 \cdot 2 \dots N}_N \cdot \underbrace{(N+1)(N+2) \dots n}_{n-N}} \leq \lim_{n \rightarrow \infty} \frac{a^N}{N!} \left(\frac{1}{2}\right)^{n-N} = 0$$

$N = 2a$

$$O(a^n) \subsetneq O(n!) \quad \forall a > 1$$

$$O(1) \subsetneq O(\log(n)) \subsetneq O(n) \subsetneq O(n^k) \subsetneq O(a^n) \subsetneq O(n!)$$

n	$\log_2 n$	n	n^3	2^n	time	$n!$	time
10	3.3	10	10^3	10^3	μs	3.6×10^6	ms
20	4.3	20	8×10^3	10^6	ms	2.4×10^{18}	77 yr
30	4.9	30	2.7×10^4	10^9	1 s	2.7×10^{32}	10^{15} yr
60	5.9	60	2×10^5	10^{18}	36 yr	8×10^{81}	10^{65} yr

Times assume one operation per flop at 10^9 flop/s.

$\det(A)$ with Laplace Expansion requires $n!$ flops

Cramer's $A^{-1} = \frac{\text{adj}(A)}{\det(A)}$ is extremely inefficient